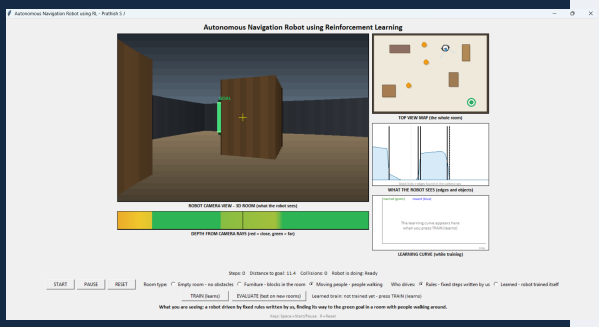
AUTONOMOUS NAVIGATION ROBOT / COMPLETE COMPUTER VISION SUBMISSION

Vision-Based Robot Navigation

Written report: pseudocode, achieved-output screenshots and implementation.

Program output: the room through the robot camera



3D room view, top view map, vision panel and depth strip in one window

Contents

System and defined task	2
Sensing: rays, edges and detections	3
The two drivers	4
Training the learned driver	5
Achieved outputs: three runs	6-8
Safety supervisor and rescue path	9
Pseudocode: three algorithms	10-12
Results	13
Implementation and limitations	14

Video explanation - separate assignment attachment

Demo video recorded from the program itself: both drivers driving, live training with the learning curve, and an evaluation run on new rooms.

Upload the demo video file alongside this PDF in the same assignment.

The pseudocode and screenshots are combined in this report. The program is a single Python file (robot_final.py) with its trained brain in qtable.json; both files are submitted with this report. Nothing needs to be installed to run it.

AUTONOMOUS NAVIGATION ROBOT / SYSTEM AND DEFINED TASK

From camera rays to driving decisions

Motivation: make a room drivable by a robot that sees only through its camera, and compare hand-written driving rules with driving the robot learns by itself.

The task is fixed and narrow. The robot starts at a fresh position in a freshly arranged rectangular room of 22 x 15 units. Somewhere in the room stands a goal, drawn as a green charging dock. The robot has reached the goal when its centre comes within 0.9 units of the dock. It must get there without driving through furniture or people. No map, no position and no route is given to the drivers while they drive. A robot that can do this holds the base skill behind warehouse robots, delivery robots and self-driving systems, and this project studies both ways of getting that skill: writing the rules by hand, and letting the robot learn them through reinforcement learning.

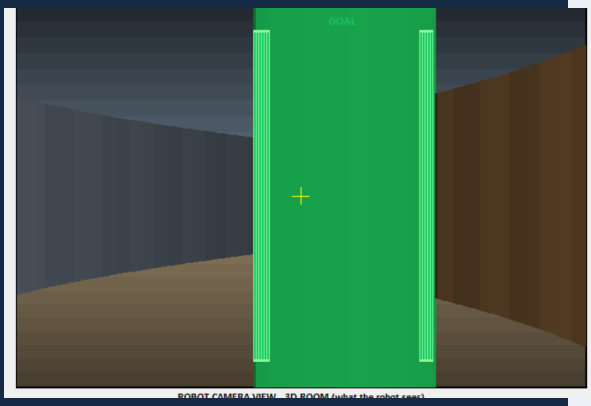
Item	Defined value
Decision input	12 camera rays: distance and hit kind (wall, furniture, person, goal, none)
Decision rate	One action every 0.12 s of robot time; 4 physics sub-steps per action
Actions	FORWARD (0.31 units), LEFT (turn 0.30 rad), RIGHT (turn 0.30 rad)
Display view	216 raycast columns, 90 degree field of view, drawn only from camera rays
Robot body	Circle, radius 0.35 units; slides along surfaces on contact
Room types	Empty room (0 blocks) / Furniture (6 blocks) / Moving people (4 blocks, 3 people)

Every run starts in a fresh room arrangement with a fresh robot position, so nothing in the driving can be memorised from a previous run. The window shows everything at once: the 3D room as the robot sees it, a top view map (drawn for the human viewer; the drivers never use it), the vision panel with edges and detections, a depth strip, the live status in plain words, and the learning curve while training runs.

Sensing: rays, edges and detections

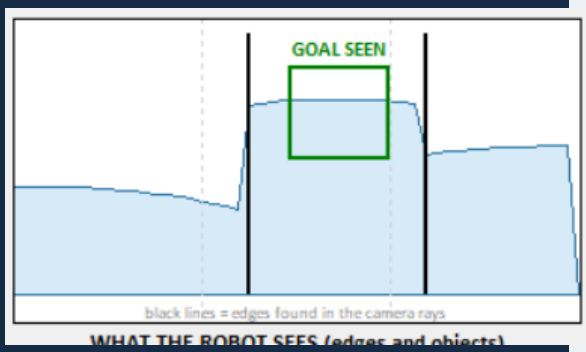
Ray evidence is kept separate from driving decisions. The drivers never see the map, the room layout or the robot coordinates.

Camera view: raycast room



216 columns; strip height from ray distance; goal pillar in green

Vision panel: saved program output



Depth profile, edge lines and the GOAL SEEN detection box

Display rays: 216 across 90 degrees

Each display ray is marched through the room until it meets a wall, a furniture block, a person or the goal, up to a range cap of 26 units. The hit distance sets the height of one vertical strip: near surfaces are tall, far surfaces are short. Strips are shaded by distance and by which face was hit; walls and furniture use different tones; people are drawn afterwards as standing figures, hidden correctly behind nearer strips by keeping one depth value per column. The goal is drawn as a bright green pillar only in the columns where it is truly in front of what is behind it.

Decision rays: 12, in three sectors

Driving uses 12 rays at equal angles. Their distances are averaged into left, middle and right sectors. Two versions of each sector average are kept: the plain average, and an obstacle average that leaves out rays hitting the goal. Avoidance reads only the obstacle averages, because the dock itself must never be treated as an obstacle; an earlier build made exactly that mistake and the robot dodged its own goal at about 1.3 units and circled forever.

Edges and the goal detection

Where the distance jumps sharply between neighbouring display rays, the vision panel draws a black edge line: those jumps are block corners and the sides of people. The goal counts as seen only when three checks all pass: bearing within 45 degrees of the heading, distance at most 13.0 units, and a clear straight line-of-sight test to the dock. Passing draws the GOAL SEEN box; a red-to-green depth strip under the camera view shows the middle distances per direction.

The two drivers: rules and Q-learning

Two drivers share the same sensing and the same three actions. One is written by hand; the other is learned, and nothing about the room is shared between them.

Rules driver

The rules driver is a short ordered list of hand-written checks, run once per decision. If a person is close and the obstacle distance in front is under 0.9 units, it turns in place towards the freer side instead of pushing forward. Otherwise, if the goal is seen, it turns towards the goal bearing (left, front or right sector) and drives forward when the way ahead is clear. If the goal is not seen, it drives forward while the front is clear and steers towards whichever side sector shows more free space, which makes it sweep the room instead of spinning. Turns are committed for a few steps once started, so the robot does not shake between left and right when two sides are almost equal.

Learned driver: Q-learning over a discretised vision state

The learned driver keeps a Q-table: one row per situation, one value per action. The situation (state) is built only from the sensing, packed into six parts:

State part	Values
Goal seen flag	0 = not seen, 1 = seen (bearing in view, within 13.0 units, line of sight clear)
Goal bearing sector	4: left (bearing over +0.3 rad), ahead (within 0.3 rad), right (under -0.3 rad), behind (over 2.6 rad either way)
Free space, left / middle / right	3 levels each from the sector distances: near (under 1.2), middle (under 3.5), far (3.5 and over)
Distance to goal	3 ranges: under 5.0, under 12.0, 12.0 and over
Total states	2 x 4 x 3 x 3 x 3 x 3 = 648 possible states; the shipped table holds values for 439 of them

Actions are FORWARD, LEFT and RIGHT. There is no WAIT action: in early training the robot discovered that standing still collects only the small step penalty and never a collision penalty, and some runs ended with a robot that politely waited forever. Removing WAIT from the learned action set ended that failure completely.

Reward, given after every action: **+100** when the goal is reached; **-10** on a collision; **-0.1** per step; plus **2.0 times** the change in distance to the goal, so moving closer pays and moving away costs. The Q-value update uses learning rate 0.2 at the start of training and a discount factor of 0.98.

Training the learned driver

Training is checked on rooms the robot never trains on, and the best table found is the one that is kept.

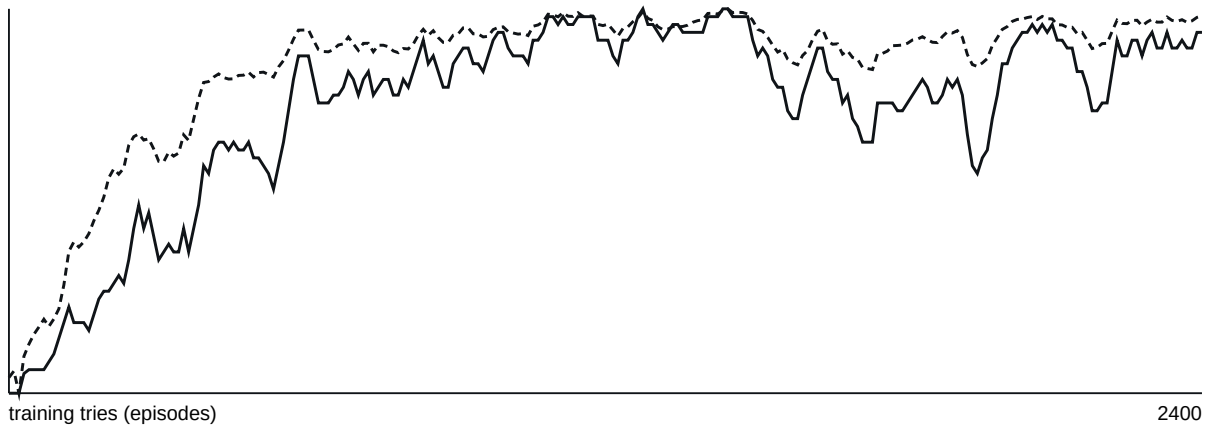
One press of the TRAIN button runs 2400 training episodes inside the program, with the learning curve drawn live. Each episode places the robot in a fresh room and lets it drive with the learned policy, except that with probability epsilon it takes a random action instead, so it keeps discovering new situations. Episodes end at the goal or after 300 steps. Room types are mixed by a curriculum that shifts as training goes on, so easy rooms never fully disappear (training one type in a solid block made the robot forget the earlier types):

Training stage	Empty / Furniture / Moving people mix
First 30 percent of episodes	60 / 30 / 10
Middle, up to 65 percent	30 / 45 / 25
Final stage	25 / 40 / 35

Exploration epsilon starts at 1.0 (fully random) and falls in a straight line to 0.05 over the first 75 percent of training, then stays at 0.05. The learning rate falls from 0.2 to 0.1 over the whole run. Every 100 episodes the current table is frozen and scored on a fixed set of test rooms with exploration off. The score prefers, in order: success on empty rooms, then mean success over the three room types, then fewer collisions, then fewer steps. The best-scoring table is snapshotted, and training ends holding that best table, never just the last one. That table is saved to qtable.json and loaded again automatically at the next start.

Training: measured learning curve (one full run, 2400 episodes)

Solid line: reached the goal, share of last 50 tries - 10 percent at try 10, 86 percent at try 2400.
Dashed line: average reward, last 50 tries - from -440 at try 10 to +88 at try 2400. Each line is scaled to its own range.



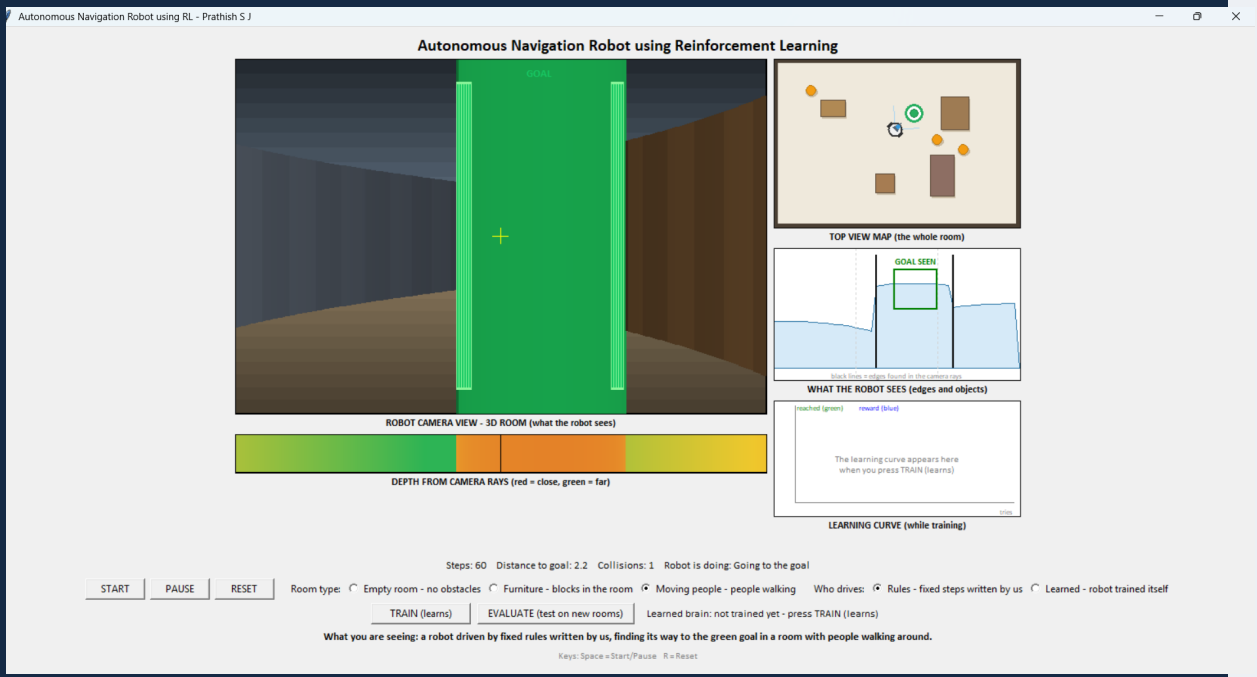
Measured run for this report, same code the TRAIN button runs. Last-50-episodes success rose from 10 percent to 86 percent; average reward rose from -440 to +88.

The final validation of that measured run, learned driver alone on fixed unseen rooms, was: empty room 100 percent, furniture 50 percent, moving people 67 percent. The gap between room types is real; it is reported again in the results and the limitations.

Achieved output: reaching the goal

60 steps taken / distance to goal 2.2 units / 1 collision / Moving people room, Rules driver

Program output: saved run on my laptop



Camera view, top view map, vision panel, depth strip and live status in one window

Final approach: the dock fills the camera view

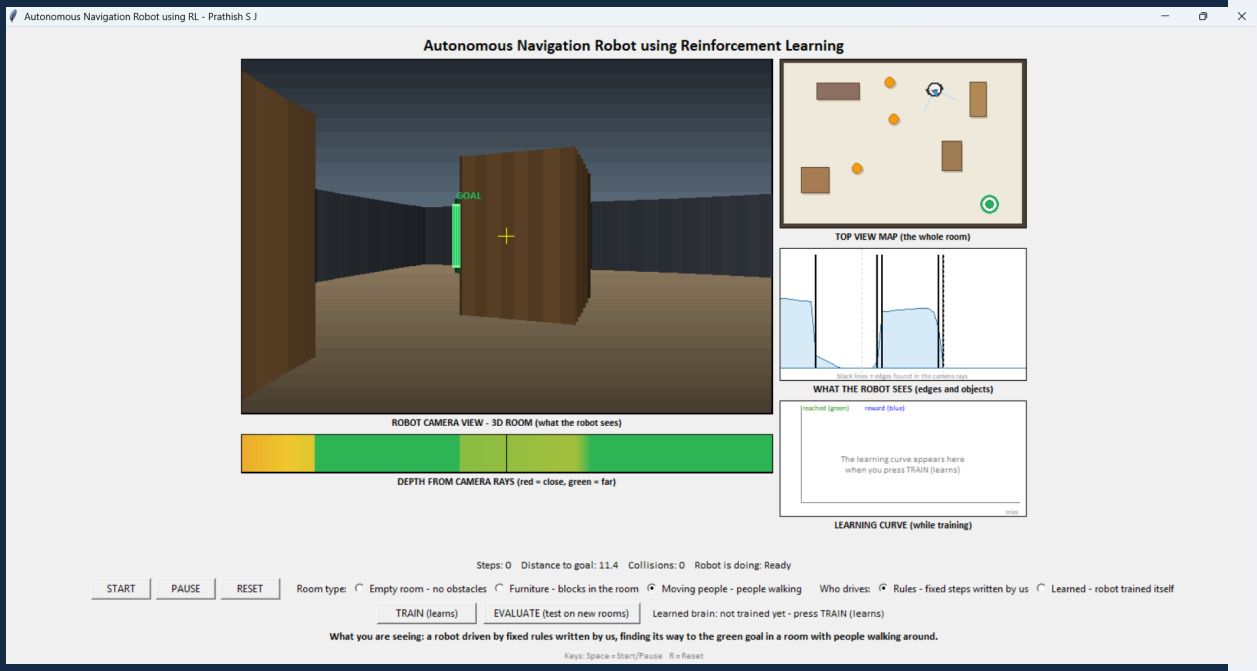
The robot is facing the goal dock almost squarely, and the dock fills the centre of the camera view because it is only 2.2 units away. The vision panel shows the GOAL SEEN box around the goal's part of the depth profile, which means all three goal checks passed at this moment: bearing inside the view, distance inside 13.0 units, and a clear line of sight.

On the top view map the robot has already crossed most of the room. Three people (orange) are walking in the room and none is inside the protected zone around the goal, so the final approach is open. The depth strip is red and orange in the middle, matching the dock and the near furniture, and green at the sides where the room is open. The status line reads Going to the goal, and the run ends in a reach a few steps after this frame.

Achieved output: a fresh room

0 steps taken / distance to goal 11.4 units / 0 collisions / Moving people room, Rules driver

Program output: saved run on my laptop



First frame of a run: a furniture block ahead, the goal visible past its corner

Start of a run: the goal is seen, but the straight way is blocked

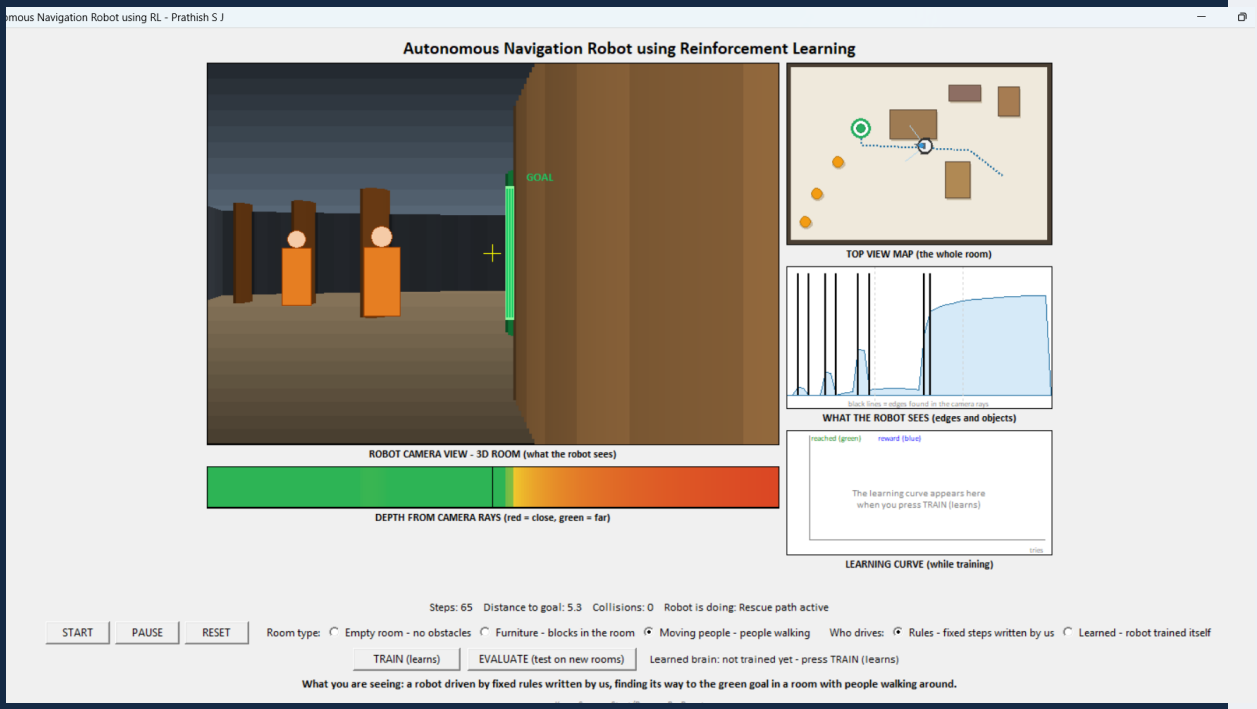
This is the first frame of a new run. The robot starts facing a large furniture block; the goal is at the far side of the room (bottom right on the top view map), 11.4 units away. In the camera view the goal is already visible as a thin bright green line at the left edge of the block: the bearing check and the line-of-sight test both pass around the block's corner, even though driving straight would hit the block.

This frame shows why goal information and obstacle information are kept separate. The goal sector says the target is to the front-left, while the obstacle averages say the front is blocked. The rules driver turns left and passes the block on that side rather than driving at the visible goal and colliding. The vision panel shows the block as a raised part of the depth profile with edge lines at both corners.

Achieved output: rescue path active

65 steps taken / distance to goal 5.3 units / 0 collisions / Moving people room, Rules driver

Program output: saved run on my laptop



The status line and the dashed path on the top view map show the rescue layer at work

Rescue output: a planned path around the block

In this frame the safety supervisor has taken over, and the screen says so: the status line reads Rescue path active. The rules driver had stopped making progress near the large block on the right, so after the escape attempt the supervisor planned a path on the room grid. The dashed blue line on the top view map is that planned path: it leads left, around the block, and straight to the goal.

Two people are visible in the camera view as orange figures. The planner marks people's current cells as blocked and re-plans every 30 steps, so the path stays valid while they move. The depth strip is deep red on the right half, matching the block face that fills the right of the camera view. The robot reached the goal on this path without a collision. This is the mechanism behind the full scores in the results section, and it is shown openly on the screen whenever it runs.

Safety supervisor and rescue path

A loop is a delay, never a dead end. The supervisor watches both drivers in the same way and answers for every run.

Camera-only drivers can loop: a hand-written rule can orbit a block, and a learned table can repeat the same action in the same state forever. The supervisor keeps three pieces of state: the best distance to the goal seen this run, the steps since that best distance last improved by at least 0.2 units, and a count of visits to coarse 0.75 unit cells of the room.

Two triggers end normal driving. First, 120 steps without progress. Second, a loop: the same coarse cell visited 8 or more times while still away from the goal. The first response is an escape: turn in place for 10 steps towards the side whose obstacle average is larger, then drive forward 5 steps if the front obstacle average is over 1.0 unit. The driver then gets a grace period of 60 steps to prove it can make progress again.

If progress still does not come, rescue mode starts. The room is converted to the planning grid: cells of 0.5 units, 44 by 30 cells, a cell blocked when its centre comes within the furniture inflated by the robot radius plus 0.2 units, or within a person's current cell when people are moving. A* search runs from the robot's cell to the goal cell, the returned path is smoothed by dropping every waypoint that can be skipped with a clear straight segment, and the robot follows the waypoints with the same FORWARD, LEFT and RIGHT actions the drivers use. The plan is rebuilt every 30 steps. If no path exists while avoiding people, the search is repeated once with people ignored, and the robot turns in place rather than driving into a person who is standing on the path.

Rescue mode hands control back only when the goal is seen, nearer than 3.5 units, and the middle rays are clear, and at most 3 times per run; after the third handoff the rescue drives all the way in by itself. Room layouts are checked with this same search when they are generated, so a static path to the goal always exists. Every rescue is counted, and the evaluation readout reports how many runs used it, which is why the results can separate what the drivers do alone from what the full system does.

Pseudocode: Algorithm 1

ALGORITHM 1: SENSE, DECIDE AND DRIVE (ONE DECISION STEP)

INPUT: robot position and heading (used by sensing only); selected driver D; Q-table if D is LEARNED

OUTPUT: one action applied to the robot; reward for the step

1. Cast the 12 vision rays across the 90 degree view; store (distance, hit kind) for each ray.

2. Average the ray distances into left, middle and right sector distances.

3. Compute obstacle averages for the same sectors, leaving out rays whose hit kind is goal.

4. Compute goal bearing and distance; set GOAL SEEN if bearing is within 45 degrees, distance is at most 13.0, and the line of sight is clear.

5. Build the state: goal seen flag, bearing sector (4), the three free-space levels (near under 1.2, middle under 3.5, far), distance range (3).

6. Ask the supervisor first: if it returns an escape or rescue action, use that action and skip to step 9.

7. IF D is RULES: run the ordered rule checks (person close, goal pursuit, free-side steering) to get the action.

8. IF D is LEARNED: look the state up in the Q-table and take the action with the highest value.

9. Apply the action over 4 physics sub-steps; on contact, slide along the surface; record whether a collision happened.

10. Move any people; bounce them off walls, reflect them off furniture, push them out of the 2.4 unit goal zone.

11. Compute the reward: +100 if within 0.9 of the goal, -10 if a collision happened, -0.1 for the step, plus 2.0 times the distance gained towards the goal.

12. Tell the supervisor the new distance so it can track progress, loops and triggers.

13. Redraw the camera view, top view, vision panel and depth strip from the new sensing.

NOTE: the drivers never receive the robot coordinates; step 1 is the only place sensing happens.

Pseudocode: Algorithm 2

ALGORITHM 2: Q-LEARNING TRAINING WITH BEST-CHECKPOINT KEEPING

INPUT: total episodes 2400; learning rate 0.2 falling to 0.1; discount 0.98

OUTPUT: the best Q-table found, saved to qtable.json

1. Start with an empty Q-table. Set exploration epsilon to 1.0.

2. FOR episode = 1 to 2400:

3. Pick the room type from the stage mix (60/30/10, then 30/45/25, then 25/40/35 for empty, furniture, moving).

4. Build a fresh room of that type with a fresh robot start; run the episode for at most 300 steps.

5. At each step, sense and build the state (ALGORITHM 1, steps 1 to 5).

6. With probability epsilon take a random action; otherwise take the best action in the Q-table.

7. Apply the action, get the reward, sense the next state.

8. Update the Q-value: move it towards reward + 0.98 x best next value, by the current learning rate.

9. Lower epsilon on its straight-line schedule (to 0.05 by 75 percent of training) and lower the learning rate towards 0.1.

10. IF episode is a multiple of 100: freeze the table and score it on fixed test rooms with exploration off.

11. Score order: empty-room success, then mean success, then fewer collisions, then fewer steps.

12. IF the frozen table scores best so far: copy it as the best table and remember the episode number.

13. After episode 2400, restore the best table (not the last one) and save it to qtable.json.

14. Record the curve point every 10 episodes: success share and average reward over the last 50 episodes.

NOTE: WAIT is not in the action set; a waiting robot was an early failure mode and is excluded by design.

Pseudocode: Algorithm 3

```
ALGORITHM 3: RESCUE PATH (A* SEARCH ON THE ROOM GRID)
INPUT: room layout (furniture, people, goal); robot position
OUTPUT: a smoothed waypoint path from the robot to the goal, or no path
1. Lay a grid of 0.5 unit cells over the room (44 x 30 cells).
2. Mark a cell blocked if its centre is too close to furniture (furniture inflated by
robot radius + 0.2).
3. IF people are moving: also mark each person's current cell blocked.
4. Run A* search from the robot cell to the goal cell, cost = steps taken +
straight-line distance to the goal.
5. IF no path was found and people were being avoided: clear the people cells and run A*
once more.
6. Smooth the cell path into waypoints: keep a waypoint only when the straight segment
to the next one is not clear.
7. Steer to the current waypoint with FORWARD, LEFT and RIGHT actions; turn in place
when a person blocks the segment.
8. Rebuild the path every 30 steps while rescue mode is active.
9. Hand control back to the driver only when the goal is seen, within 3.5 units, and the
middle rays are clear; allow at most 3 such handoffs, then drive in fully.
NOTE: room layouts are checked with this same search when they are generated, so a
static path to the goal always exists.
```

Results

Measured on 30 unseen rooms per setting, supervisor on and off, with a 1500 step limit per run.

The program's EVALUATE mode runs the chosen driver on 20 brand new rooms. For this report a larger test was run with the same code: 30 unseen rooms for every combination of room type and driver, supervisor on, which is the normal configuration. A run counts as reached when the robot comes within 0.9 units of the goal. A run is reported as stuck only if it ends still far from the goal; none did.

Driver and room	Reached	Average steps	Runs that used rescue
Rules - Empty room	30 of 30	45	0 of 30
Rules - Furniture	30 of 30	71	16 of 30
Rules - Moving people	30 of 30	69	14 of 30
Learned - Empty room	30 of 30	50	0 of 30
Learned - Furniture	30 of 30	88	12 of 30
Learned - Moving people	30 of 30	78	9 of 30

On empty rooms both drivers also score near-perfect path efficiency (SPL 1.00 for rules, 0.97 for learned). Training repeatability was checked separately: three full fresh trainings, each followed by the same 30-room test with the supervisor on, again reached 30 of 30 in every room type.

The same test with the supervisor off shows what the drivers do alone, and it is weaker. The learned driver alone reached 29 of 30 empty rooms, 16 of 30 furniture rooms and 24 of 30 moving rooms. The rules driver alone, on a smaller 10-room sample, reached 10 of 10, 7 of 10 and 6 of 10. The learned table's weak spot is furniture: several different real situations pack into the same discretised state there, so one stored action has to serve all of them. The full-system scores above are therefore reported as system scores, with the rescue counts printed next to them, not as scores of the learned driver alone.

Implementation and limitations

What was built, what it assumes, and where it stops. Observed sensing, driver decisions and learned values are different kinds of evidence and are kept separate.

Raycast vision

216 display rays and 12 decision rays per step; per-column depth buffer for people sprites; edges from depth jumps; goal detection gated by bearing, range and line of sight.

Continuous motion and collision

The world is continuous, not a grid: the robot is a circle moved in 4 sub-steps per decision and pushed out of rectangles along the shortest exit, which produces wall sliding. People are circles with constant velocity, wall bouncing and reflection off furniture.

Q-learning

Tabular Q-learning over the 648-state discretisation, epsilon-greedy exploration, curriculum mixing of room types, fixed-room validation every 100 episodes and best-checkpoint saving to qtable.json.

Safety supervisor

Progress window of 120 steps, loop detection on 0.75 unit cells, a scripted escape turn, then A* rescue on a 0.5 unit grid with re-planning every 30 steps and limited handoffs back to the driver.

Interface

One window: 3D camera view, top view map with the rescue path drawn when active, vision panel, depth strip, live stats in plain words, and START, PAUSE, RESET, TRAIN and EVALUATE controls. Room types and drivers are labelled in plain words, with a one-line explanation of what is currently being shown.

Limitations. The room, the people and the camera are simulated: people move in straight lines and turn at surfaces; they are not detected or predicted from images. The learned driver is a table over a small state, not a neural network, and by itself it is weak in furniture rooms; the rescue layer that covers this plans on the room grid, which the drivers themselves never see. Nothing here measures performance on a real robot, a real camera or a real room.

Models and files. No pretrained models are used anywhere. The only learned component is the Q-table, trained from scratch inside the program; the shipped file (format marker room3d-v1) holds values for 439 visited states. Program: robot_final.py, a single Python file using only the standard library (tkinter, math, random, json, os, heapq). Project: Autonomous Navigation Robot using Reinforcement Learning, Foundations of Computer Vision, October 2026.